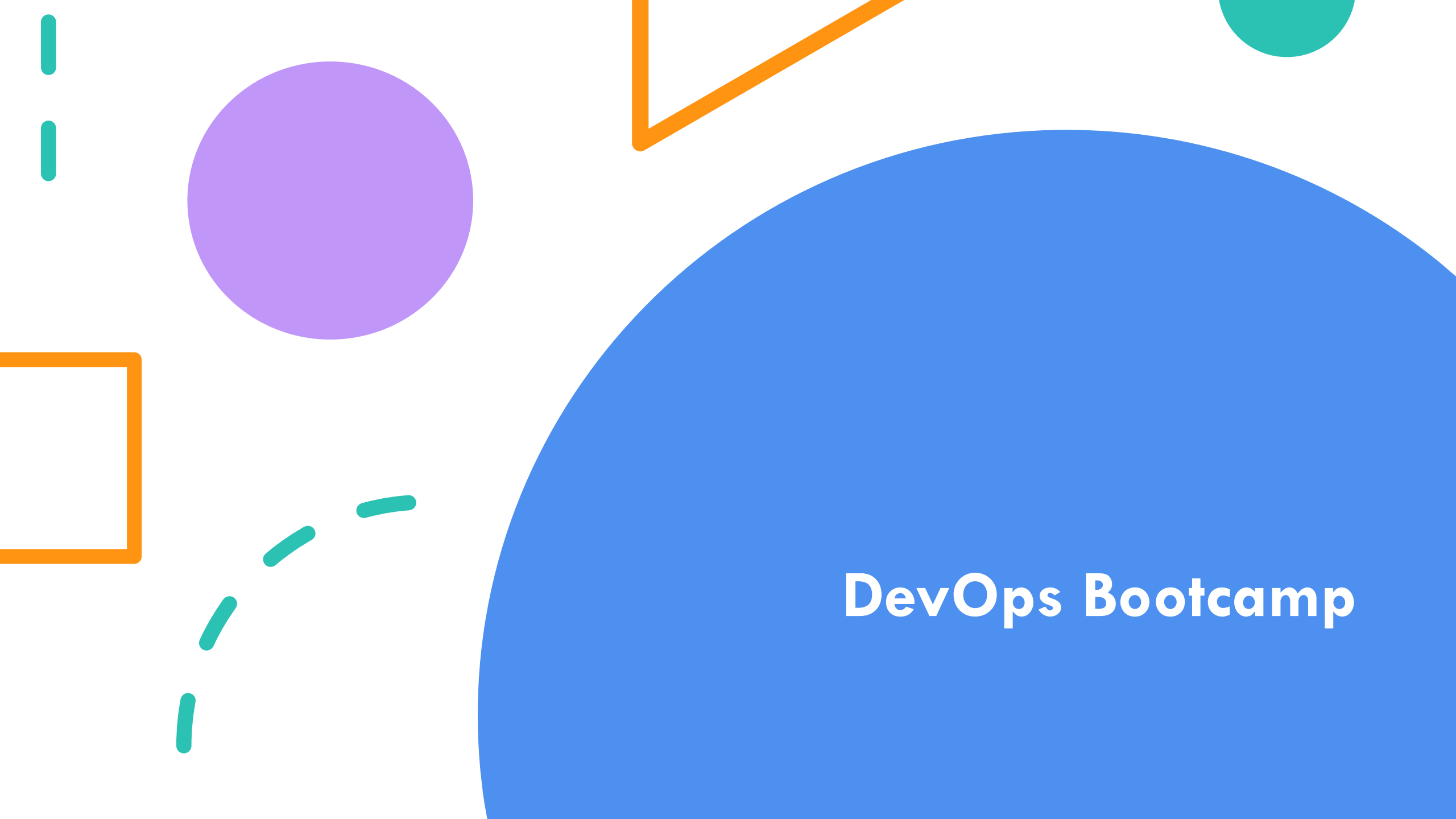
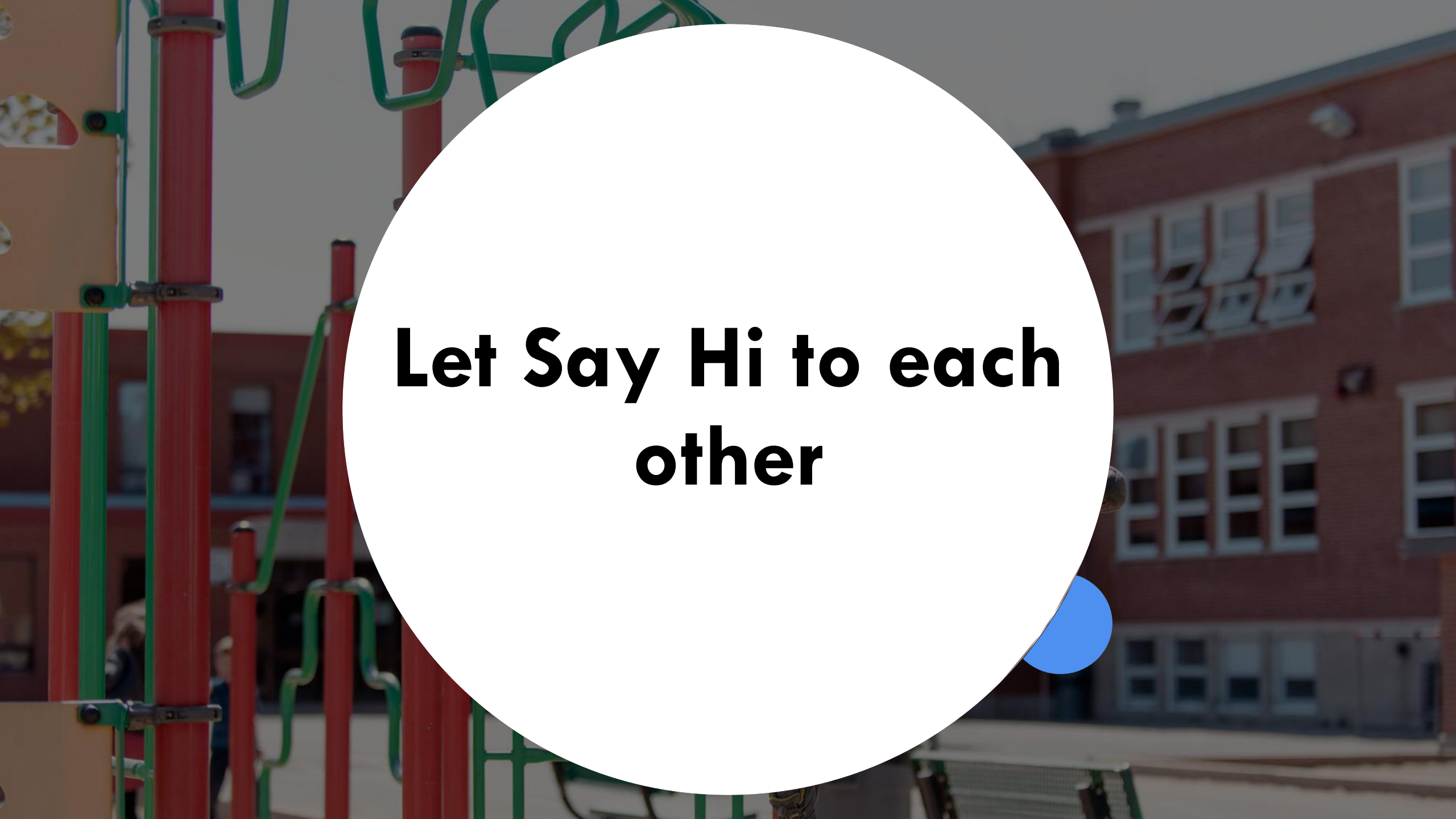
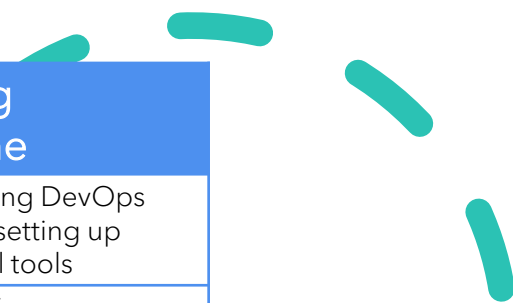




DevOps Bootcamp



**Let Say Hi to each
other**



Module	Focus Area	Lessons	Key Technologies	Learning Outcome
Module 1	Foundation & Overview	1	DevOps principles, development environment	Understanding DevOps culture and setting up foundational tools
Module 2	Linux & Version Control	2-4	Linux, Shell scripting, Git, GitHub	Mastery of Linux administration and version control workflows
Module 3	Containerization & Orchestration	5-8	Docker, Kubernetes, Helm	Container technology and orchestration for scalable applications
Module 4	Cloud Computing	9-10	AWS EC2, S3, VPC, IAM, Load Balancers	Cloud infrastructure design and implementation
Module 5	CI/CD Implementation	11-12	Jenkins, Pipeline as Code, Automation	Automated build, test, and deployment pipelines
Module 6	Monitoring & Observability	13-14	Prometheus, Grafana, Loki, Alerting	Comprehensive monitoring, logging, and observability solutions
Module 7	Capstone Project	15-16	Integration of all technologies	Real-world project demonstrating end-to-end DevOps implementation



Today's Lesson



What is DevOps?

DevOps Culture & Principles

DevOps Lifecycle

Tools Ecosystem Overview

Lab preparation

What is DevOps

- DevOps combines development (**Dev**) and operations (**Ops**) to increase the efficiency, speed, and security of software development and delivery compared to **traditional processes**. A more nimble software development lifecycle results in a competitive advantage for businesses and their customers.



Development

Develop software including: Planning, Design, Architect, Coding and Test,

Operation

Provisioning infrastructure, development environment, deploy, release software, and monitoring, maintain, operate software

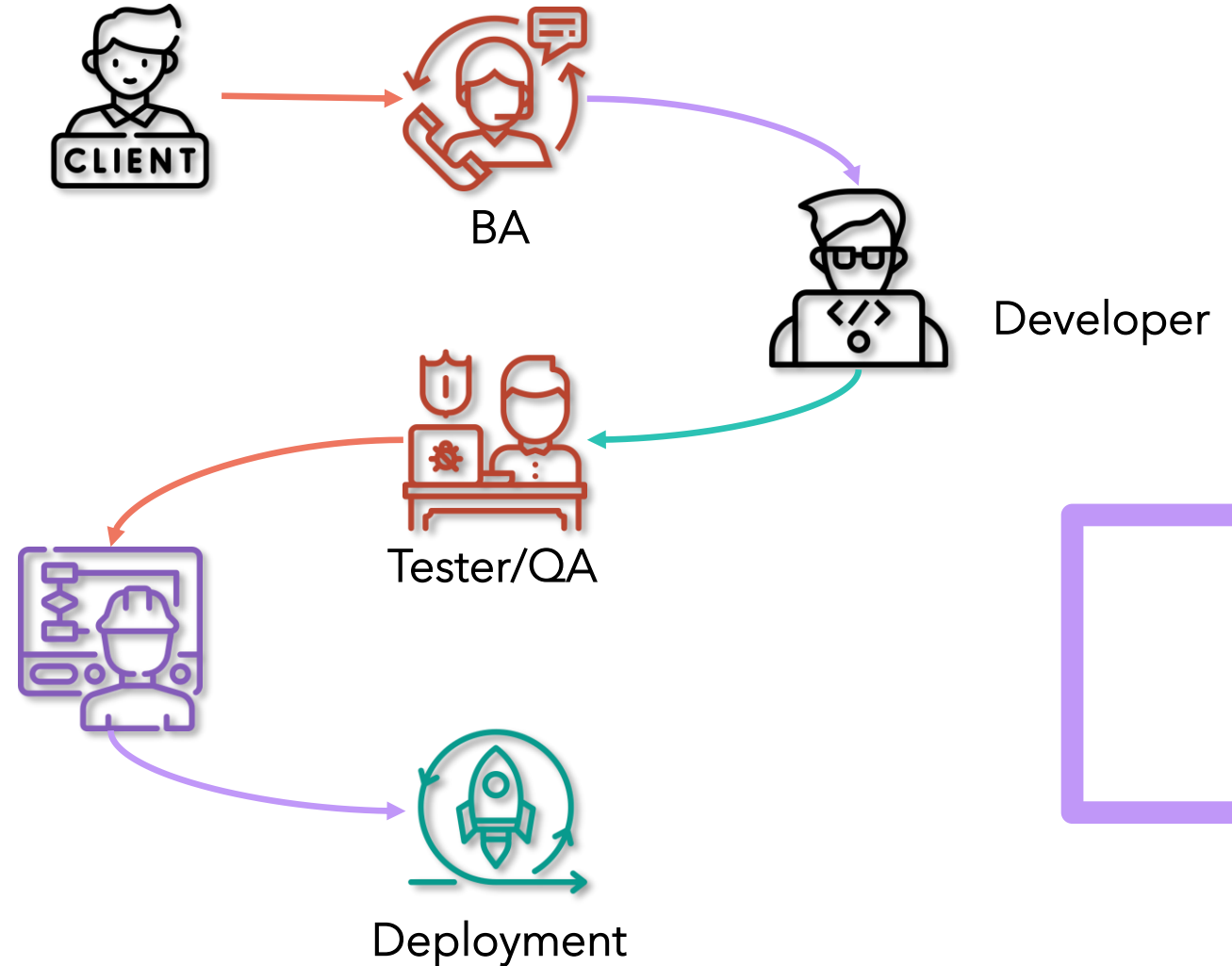


Still Confused?

Example for **traditional** software development

- BA collect requirement from client and request to developer
- Developer start coding and send it to tester
- Tester test it and mark it ready for deployment
- Operator deploy the version of software which qualified for release

Operator



Downside for traditional software development

⌚ Longer Development Cycles

Projects move through fixed phases (requirements → design → coding → testing → deployment), so working software appears late in the process. Delays in one stage ripple through the rest, pushing back delivery.

💰 Higher Costs

Extensive upfront planning and documentation require significant time and resources.

Late discovery of defects means fixes are more expensive than if caught earlier.

🔒 Lack of Flexibility

Once requirements are locked in, changes are difficult and costly to implement. This rigidity can be a poor fit for fast-changing business or market needs.

👤 Limited Customer Involvement

Clients or end users are often only engaged during the initial requirements phase and final delivery.

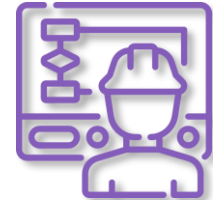
This can lead to a product that technically meets the spec but misses evolving user expectations.

🕒 Slow Feedback Loop

Testing is typically done after the build is complete, so feedback on usability or performance comes too late to make major adjustments efficiently.

⚙️ Maintenance Complexity

Large, monolithic codebases from traditional projects can be harder to maintain and scale over time.



How DevOps improve this?

DevOps directly tackles many of the pain points of traditional software development by blending **culture, process, and tooling** to make delivery faster, more reliable, and more adaptable



And the **outcome**

Shorter Development Cycles

Continuous Integration & Continuous Delivery (CI/CD) means code is integrated, tested, and deployed in small, frequent batches rather than big, infrequent releases.

This reduces the “big bang” risk of late delivery and gets features to users faster.

Breaking Down Silos

DevOps unites development and operations into **cross-functional teams** with shared goals.

Greater Flexibility & Responsiveness

Frequent releases make it easier to adapt to changing requirements or market conditions without derailing the entire project.

Feedback from users can be incorporated quickly into the next iteration.

Automation Everywhere

Automated testing, deployment, and infrastructure provisioning (Infrastructure as Code) reduce human error and speed up delivery.

This also ensures consistency across environments, making releases more predictable.

Faster Feedback Loops

Continuous monitoring and automated alerts mean issues are detected and resolved earlier in the cycle.

Improved Reliability

Practices like automated rollback, blue-green deployments, and canary releases reduce downtime and risk during updates.

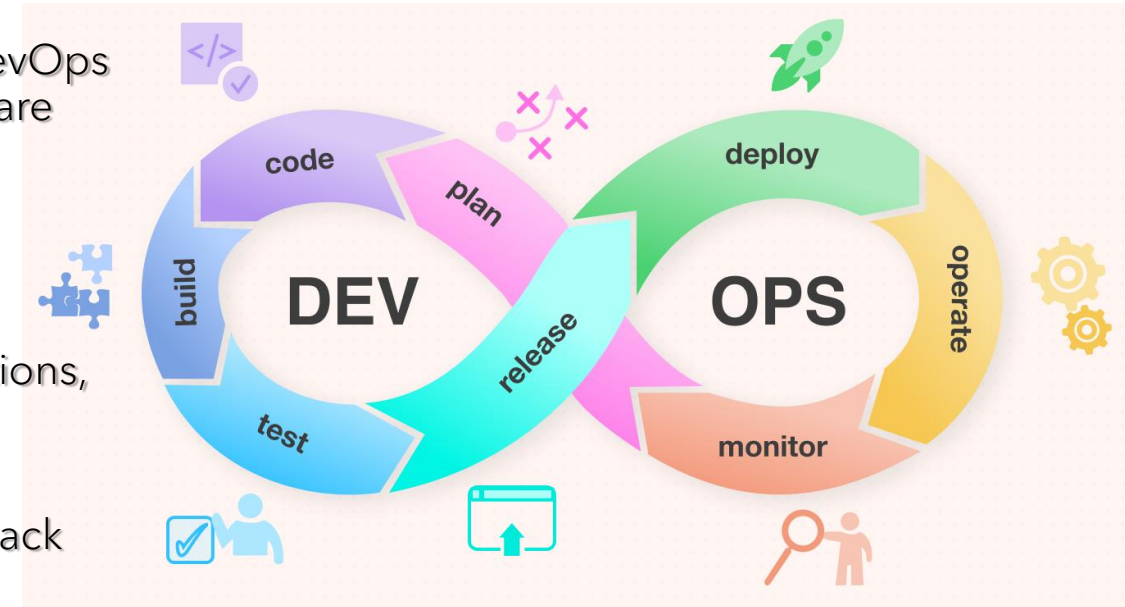


What is DevOps

DevOps is a modern approach to building and running software that blends **development (Dev)** and **operations (Ops)** into a single, collaborative process. Instead of treating coding, testing, deployment, and maintenance as separate, siloed activities, DevOps unites people, processes, and tools so teams can deliver software **faster, more reliably, and with continuous improvement**

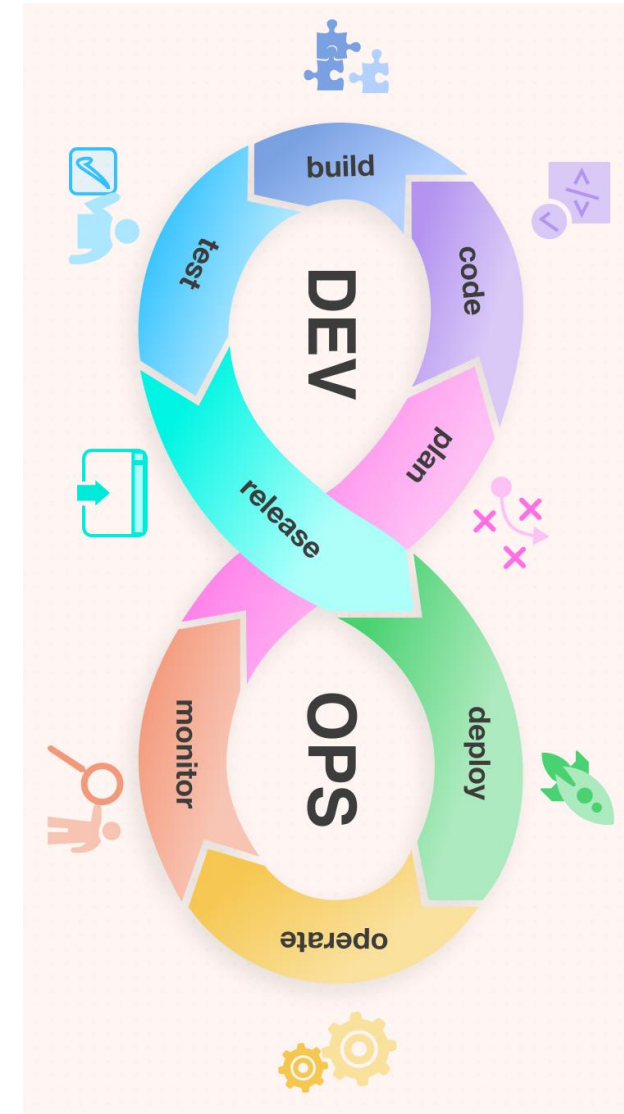
Core Idea

- **Culture:** Break down barriers between developers, IT operations, QA, and security so everyone shares responsibility for the product's success.
- **Process:** Use iterative, agile-style workflows with rapid feedback loops.
- **Technology:** Automate as much as possible – from code integration to infrastructure provisioning.



What is DevOps Engineer Do?

A **DevOps engineer** is the bridge-builder between software development and IT operations – part coder, part systems architect, part automation wizard, and part problem-solver. Their mission is to make sure software moves from idea to production quickly, reliably, and continuously.



What skills DevOps need?

Category	Skills to Learn	Popular Tools / Technologies
OS & Scripting	Linux fundamentals, shell scripting, Python, Go	Bash, Python, Go
Networking & Security	DNS, HTTP/HTTPS, load balancing, IAM, SSL/TLS, firewalls	Nmap, Wireshark, Vault
Version Control	Branching, merging, Git workflows, GitOps	Git, GitHub, GitLab, Bitbucket
CI/CD	Automated builds, testing, deployment, rollback strategies	Jenkins, GitHub Actions, GitLab CI, CircleCI, ArgoCD
Containerization	Build, run, manage containers	Docker, Podman
Orchestration	Container scheduling, scaling, service discovery	Kubernetes, OpenShift, Nomad, Helm
Infrastructure as Code (IaC)	Provisioning, automation, configuration management	Terraform, Ansible, Pulumi, CloudFormation
Cloud Platforms	Compute, storage, networking, managed services	AWS, Azure, GCP
Monitoring & Logging	Metrics, alerts, visualization, log analysis	Prometheus, Grafana, ELK Stack, Loki, Splunk
Automation & Config Mgmt	System configuration, patch management, repeatable automation	Ansible, Puppet, Chef, SaltStack
Collaboration & Agile	Team communication, issue tracking, documentation	Jira, Confluence, Slack, Microsoft Teams
Soft Skills	Collaboration, problem-solving, communication, incident response	N/A

Is that A LOT?

Category	Skills to Learn	Popular Tools / Technologies
OS & Scripting	Linux fundamentals, shell scripting, Python, Go	Bash, Python, Go
Networking & Security	DNS, HTTP/HTTPS, load balancing, IAM, SSL/TLS, firewalls	Nmap, Wireshark, Vault
Version Control	Branching, merging, Git workflows, GitOps	Git, GitHub, GitLab, Bitbucket
CI/CD	Automated builds, testing, deployment, rollback strategies	Jenkins, GitHub Actions, GitLab CI, CircleCI, ArgoCD
Containerization	Build, run, manage containers	Docker, Podman
Orchestration	Container scheduling, scaling, service discovery	Kubernetes, OpenShift, Nomad, Helm
Infrastructure as Code (IaC)	Provisioning, automation, configuration management	Terraform, Ansible, Pulumi, CloudFormation
Cloud Platforms	Compute, storage, networking, managed services	AWS, Azure, GCP
Monitoring & Logging	Metrics, alerts, visualization, log analysis	Prometheus, Grafana, ELK Stack, Loki, Splunk
Automation & Config Mgmt	System configuration, patch management, repeatable automation	Ansible, Puppet, Chef, SaltStack
Collaboration & Agile	Team communication, issue tracking, documentation	Jira, Confluence, Slack, Microsoft Teams
Soft Skills	Collaboration, problem-solving, communication, incident response	N/A

Yes, That is A LOT 😊



Don't Worry

The magic isn't in knowing *every* tool – it's in **mastering the principles** (automation, feedback loops, scalability) so you can **adapt** to whatever stack your team uses.

What will we learn in this course

- We'll explore nearly every facet of DevOps, following a streamlined learning path from start to finish
- By the end, you'll work on a real-world project designed to ensure you feel fully confident applying DevOps in practice.

Module	Focus Area	Lessons	Key Technologies	Learning Outcome
Module 1	Foundation & Overview	1	DevOps principles, development environment	Understanding DevOps culture and setting up foundational tools
Module 2	Linux & Version Control	2-4	Linux, Shell scripting, Git, GitHub	Mastery of Linux administration and version control workflows
Module 3	Containerization & Orchestration	5-8	Docker, Kubernetes, Helm	Container technology and orchestration for scalable applications
Module 4	Cloud Computing	9-10	AWS EC2, S3, VPC, IAM, Load Balancers	Cloud infrastructure design and implementation
Module 5	CI/CD Implementation	11-12	Jenkins, Pipeline as Code, Automation	Automated build, test, and deployment pipelines
Module 6	Monitoring & Observability	13-14	Prometheus, Grafana, Loki, Alerting	Comprehensive monitoring, logging, and observability solutions
Module 7	Capstone Project	15-16	Integration of all technologies	Real-world project demonstrating end-to-end DevOps implementation